DOCUMENTACIÓN TÉCNICA - SIMULADOR EPIDEMIOLÓGICO

Proyecto Final IPC2

Curso: Introducción a la Programación y Computación 2 Sección D
Carnet: 202300583
Nombre: Angel Mazariegos
Semestre: Primer Semestre 2025
Fecha: 5 de Marzo de 2026

---

1. INTRODUCCIÓN

Para este proyecto tuve que hacer un simulador que modela cómo se propaga una enfermedad en una población. La idea es usar un autómata celular en una matriz de M×M celdas, donde cada celda puede estar sana (0) o contagiada (1).

Lo más importante fue que NO podía usar ninguna colección nativa de C# como List<T>, LinkedList<T>, Dictionary o arrays 2D [,]. Todo tuvo que hacerse con estructuras de datos propias (TDA).

Qué hace el sistema

Básicamente el programa hace esto:

- Lee un archivo XML con información de pacientes y su rejilla inicial
- Simula cómo evoluciona la enfermedad período por período
- Muestra la rejilla en consola con caracteres ASCII
- Detecta si hay patrones que se repiten para dar un diagnóstico (Leve, Grave o Mortal)
- Exporta los resultados a otro archivo XML

---

2. OBJETIVOS

Objetivo Principal

Hacer un simulador que pueda analizar cómo se propaga una enfermedad y clasificarla según su gravedad basándose en cómo evoluciona con el tiempo.

Objetivos Específicos

1. Crear una matriz con punteros en 8 direcciones (vecindad de Moore) sin usar arrays normales
2. Aplicar las reglas de contagio y supervivencia según los vecinos de cada celda
3. Guardar el histórico de estados para detectar si hay patrones que se repiten
4. Hacer una interfaz donde se pueda ver paso a paso o automático
5. Leer y escribir archivos XML para entrada y salida de datos

---

3. DISEÑO DEL SISTEMA

Organicé el código en varias partes para que fuera más fácil de entender:

Estructuras de Datos Base (TDA)

- Nodo<T>: El nodo básico para las listas enlazadas
- ListaEnlazada<T>: Mi lista enlazada propia con métodos como Insertar, Buscar, etc.
- NodoMatriz<T>: Un nodo especial con 8 punteros (para los vecinos)
- Matriz<T>: La matriz completa hecha con punteros en lugar de arrays

Clases del Dominio

- Celda: Guarda la fila, columna y estado de una celda
- Paciente: Tiene los datos del paciente y su rejilla inicial
- DiagnosticoResultado: Donde guardo el resultado (Leve/Grave/Mortal y los valores N y N_1)

Lógica Principal

- Automata: La clase que hace toda la simulación
    - Inicializa la matriz con el paciente
    - Aplica las reglas período por período
    - Busca patrones en el histórico
    - Calcula el diagnóstico

Interfaz de Usuario

- Program: El menú y todo lo que ve el usuario
    - Menú con 7 opciones
    - Lectura y escritura de XML
    - Visualización con □ y ·
    - Dos modos: paso a paso o automático

---

4. ESTRUCTURAS DE DATOS

4.1 ListaEnlazada

Esta es la estructura que más usé en todo el proyecto. Básicamente es una lista enlazada normal pero hecha desde cero.

Propiedades:

- Cabeza: Apunta al primer nodo
- Tamaño: Lleva la cuenta de cuántos elementos hay

Métodos importantes:

- Insertar(T dato): Agrega al final (tarda O(n) porque hay que llegar hasta el último)
- ObtenerPorIndice(int indice): Busca por posición (O(n) también)
- ParaCada(Action<T> accion): Recorre toda la lista haciendo algo con cada elemento
- BuscarPrimero(Predicate<T> condicion): Encuentra el primer elemento que cumpla algo
- Existe(Predicate<T> condicion): Verifica si existe un elemento que cumpla la condición
- Limpiar(): Borra todo
- EstaVacia(): Checa si está vacía

4.2 Matriz Ortogonal

Aquí es donde se puso interesante el proyecto. Tenía que hacer una matriz 2D pero SIN usar new int[M, M] porque está prohibido.

Cómo la construí:

1. Creo M×M nodos (cada uno es un NodoMatriz)

2. Los guardo en ListaEnlazada<ListaEnlazada<NodoMatriz<T>>>

3. Conecto cada nodo con sus vecinos:

  - Arriba, Abajo, Izquierda, Derecha (los ortogonales)

  - ArribaIzquierda, ArribaDerecha, AbajoIzquierda, AbajoDerecha (las diagonales)

4. Guardo la referencia al nodo (0,0) como Cabeza

Complejidad:

- Construir la matriz: $O(M^2)$ - tiene sentido porque son M×M nodos
- Llegar a una celda específica: O(M) porque hay que navegar desde Cabeza
- Obtener vecinos: O(1) porque ya tengo los punteros directos

¿Por qué no simplemente usar arrays? Porque la restricción del proyecto dice que no puedo usar colecciones nativas. Además, con punteros es más eficiente para acceder a los vecinos de una celda.

---

5. CÓMO FUNCIONA EL AUTÓMATA

5.1 Inicializar el Autómata

Cuando cargo un paciente, esto es lo que pasa:

Método: Automata.Inicializar(Paciente paciente)

1. Creo una matriz M×M y pongo todas las celdas en 0 (sanas)
2. Leo las celdas contagiadas del paciente:
   - Para cada celda en paciente.Rejilla
   - Pongo esa posición en 1 (contagiada)
3. Hago una copia del estado inicial y la guardo en Historico
4. PeriodoActual = 0
5. Actualizo los contadores de cuántas están sanas y contagiadas

5.2 Simular un Período

Esto se ejecuta cada vez que avanza un período:

Método: Automata.SimularPeriodo()

1. Creo una nueva matriz vacía (para el siguiente estado)
2. Para cada celda (i,j):
   a) Cuento cuántos vecinos tiene contagiados
   b) Leo su estado actual
   c) Si está contagiada:
      - Aplico Regla 1 (sobrevive con 2-3 vecinos)
     Si está sana:
      - Aplico Regla 2 (se contagia con 3 vecinos exactos)
   d) Guardo el nuevo valor en la matriz nueva
3. Reemplazo el estado actual con la matriz nueva

4. Incremento el período

5. Guardo copia en el histórico

6. Actualizo contadores

5.3 Contar Vecinos

Este método es clave:

Método: ContarVecinosContagiados(fila, columna) → int

1. Obtengo la lista de 8 vecinos con ObtenerVecinos()

2. contador = 0

3. Recorro cada vecino:

   Si vecino == 1:

    contador++

4. Devuelvo contador

Lo bueno es que ObtenerVecinos() usa los punteros del NodoMatriz, entonces es rápido (O(1)).

5.4 Las Reglas

Regla 1:

Si estado == 1 (contagiada) Y (vecinos == 2 O vecinos == 3):

 return 1 // Sobrevive

Si no:

 return 0 // Se recupera

Regla 2:

Si estado == 0 (sana) Y vecinos == 3:

 return 1 // Se contagia

Si no:

 return 0 // Sigue sana

---

6. SISTEMA DE DIAGNÓSTICO

6.1 El Histórico

Para poder detectar patrones tengo que guardar cómo se ve la matriz en cada período.
Uso ListaEnlazada<Matriz<int>> Historico donde cada elemento es una copia completa de la matriz en ese momento.

Período 0: [Copia de la matriz inicial]

Período 1: [Copia después de aplicar reglas 1 vez]

Período 2: [Copia después de 2 veces]

...

Sí, consume memoria ($O(P \times M^2)$ donde P es períodos), pero es necesario para comparar.

6.2 Comparar Matrices

Este método simplemente compara celda por celda:

Método: CompararMatrices(Matriz m1, Matriz m2) → bool

```
1. sonIguales = true
2. Para cada celda (i,j):
   Si m1[i,j] ≠ m2[i,j]:
     sonIguales = false
3. return sonIguales
```

6.3 Evaluar el Diagnóstico

Aquí es donde evaluo si es Leve, Grave o Mortal:

Método: EvaluarDiagnostico() → DiagnosticoResultado

```
// PASO 1: Buscar si es MORTAL
Si el período actual es mayor a 0:
  estadoInicial = Historico[0]
```

Si la matriz actual es igual a la inicial:

Diagnóstico = "Mortal"

N = período actual (cuándo volvió al inicio)

$N_1 = 0$

TERMINAR

// PASO 2: Buscar si es GRAVE

Para i desde 1 hasta período actual-1:

estadoAnterior = Historico[i]

Si la matriz actual es igual a estadoAnterior:

PeríodoOscilacion = PeriodoActual - i

Diagnóstico = "Grave"

N = i (dónde empezó el patrón)

N_1 = periodoOscilacion (cada cuánto se repite)

TERMINAR

// PASO 3: Si no encontré nada, es LEVE

Diagnóstico = "Leve"

N = período actual

$N_1 = 0$

6.4 Ejemplos

MORTAL:

P0: Estado A

P1: Estado B

P2: Estado C

P3: Estado A ← Igual a P0!

Resultado: MORTAL, N=3, N_1=0

GRAVE:

P0: Estado A

P1: Estado B

P2: Estado C

P3: Estado B ← Igual a P1!

Resultado: GRAVE, N=1, N_1=2

LEVE:

P0: Estado A

P1: Estado B

P2: Estado C

P3: Estado D ← Todos diferentes

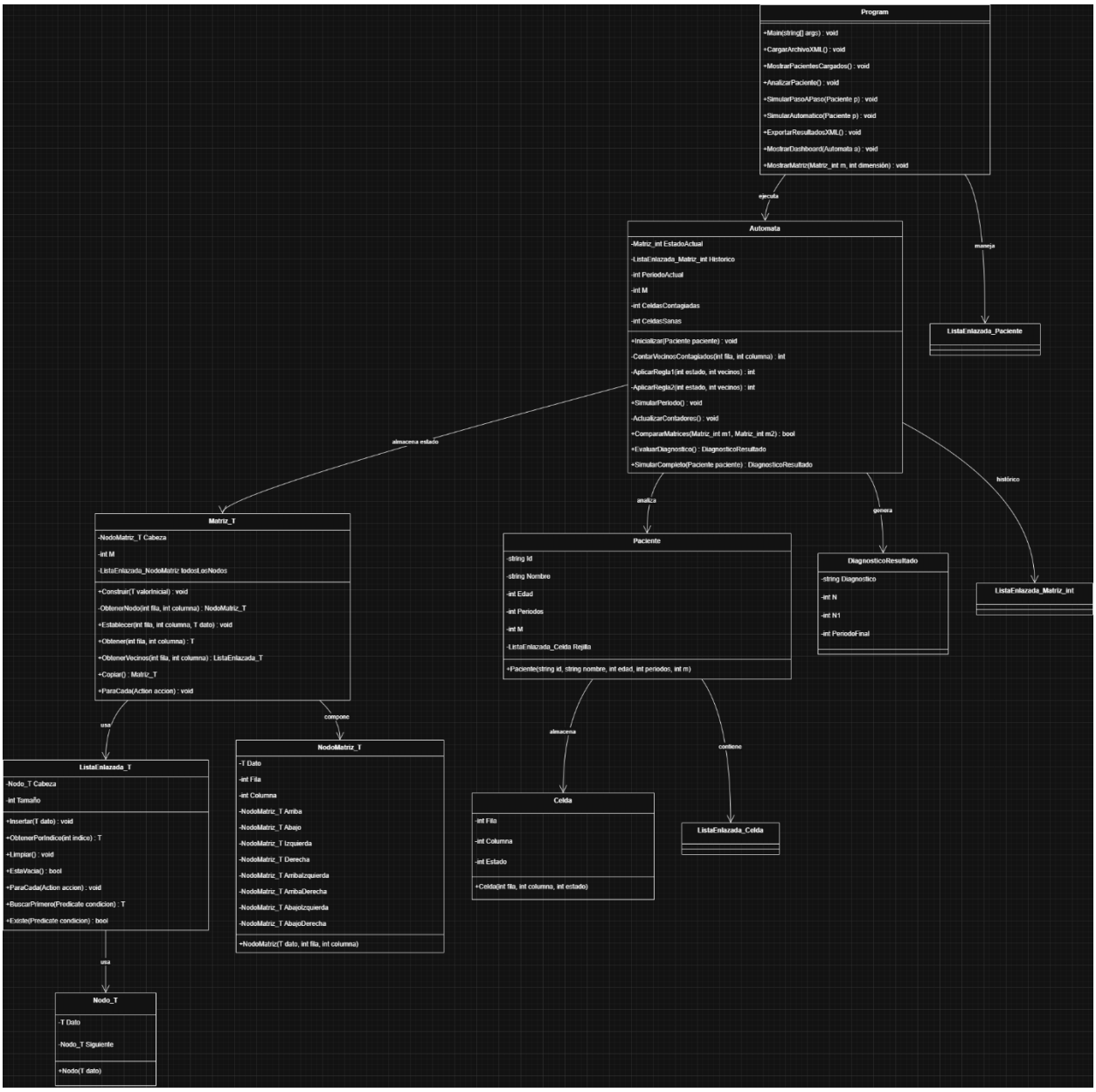
Resultado: LEVE, N=3

---

7. DIAGRAMAS

7.1 Diagrama de Clases

Este diagrama muestra todas las clases que hice y cómo se relacionan entre sí:



Cómo se conectan:

- ListaEnlazada usa Nodo
- Matriz está compuesta de NodoMatriz
- Matriz usa ListaEnlazada internamente

- Automata guarda una Matriz para el estado actual
- Automata tiene una ListaEnlazada<Matriz> para el histórico
- Automata recibe un Paciente y genera un DiagnosticoResultado
- Paciente contiene una ListaEnlazada con las celdas contagiadas
- Program maneja una ListaEnlazada con todos los pacientes cargados

---

7.2 Diagrama de Actividades

Este muestra el flujo del programa desde que abres el menú hasta que exportas resultados:

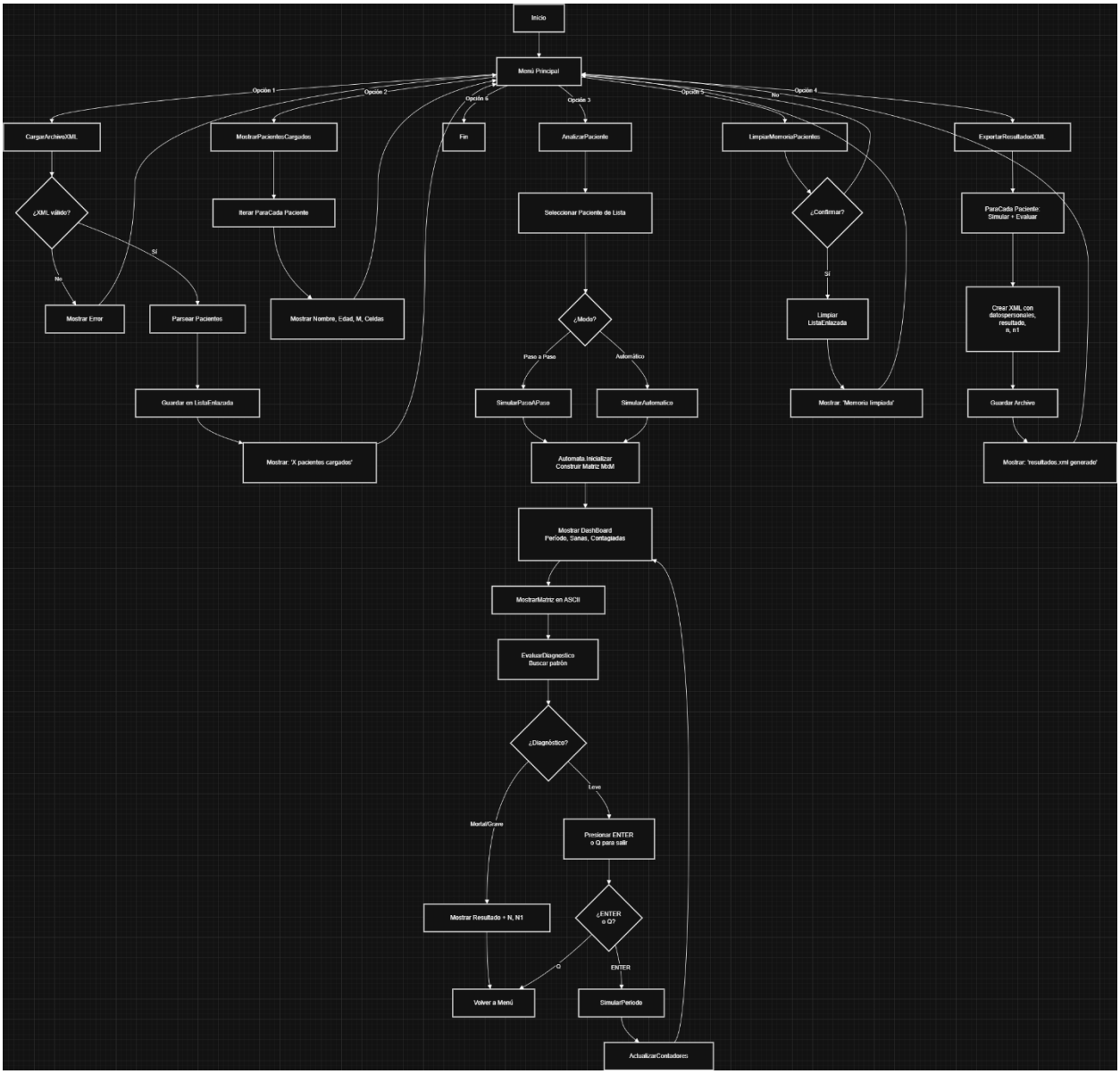


Figura 2: Flujos del sistema - cómo se carga el XML, se simula un paciente (paso a paso o automático), se evalúa el diagnóstico y se exporta todo a XML.

Qué hace cada flujo:

1. **Cargar XML:** Lee el archivo → Verifica que esté bien → Crea objetos Paciente → Los mete en la lista global
2. **Simulación Paso a Paso:** Inicializa todo → Muestra el dashboard → Dibuja la rejilla → Espera que presiones ENTER → Simula → Checa diagnóstico → Repite
3. **Simulación Automática:** Inicializa → Corre todos los períodos de jalón → Te da el resultado

4. **Exportar XML:** Recorre todos los pacientes → Simula cada uno → Arma el XML con los resultados → Lo guarda

---

8. VISUALIZACIÓN GRAPHVIZ

El sistema incluye generación automática de diagramas usando Graphviz.

8.1 Cómo Funciona

1. El usuario selecciona un paciente de la lista
2. Selecciona el período que desea visualizar (0 hasta el período final)
3. El sistema genera un archivo .dot con la estructura de la matriz
4. Se ejecuta el comando: dot -Tpng archivo.dot -o imagen.png
5. Opcionalmente abre la imagen automáticamente

8.2 Formato de Visualización

- Cada celda es un nodo cuadrado
- Color azul = Celda contagiada (estado = 1)
- Color blanco = Celda sana (estado = 0)
- Layout en cuadrícula usando rank=same por filas
- Conexiones invisibles mantienen la estructura

8.3 Restricción TDA

La generación del contenido .dot usa exclusivamente ListaEnlazada<string> para construir cada línea del archivo. NO se usan arrays, List<> ni StringBuilder nativo.

9. CONCLUSIONES

9.1 Qué logré

1. **Matriz sin arrays:** Pude hacer una estructura 2D completa usando solo nodos con punteros en 8 direcciones. Al principio pensé que sería imposible, pero funcionó bien.
2. **El autómata funciona:** Las reglas de supervivencia y contagio se aplican correctamente. Ver cómo cambia la rejilla período por período es bien interesante.
3. **Detección de patrones:** Comparar el histórico de matrices funciona para encontrar si se repiten. Fue la parte más complicada porque tuve que guardar copias de toda la matriz en cada paso.
4. **Modo paso a paso:** Ayuda mucho para entender qué está pasando. Puedes ir viendo cómo cada celda decide si se contagia o se recupera.
5. **Cumplí con la restricción:** No usé ni una sola colección nativa. TODO está hecho con mis propias estructuras de datos.
6. **Visualización Graphviz:** Se implementó la generación automática de archivos .dot que crean imágenes PNG con la cuadrícula coloreada (azul=contagiado, blanco=sano). El usuario puede seleccionar cualquier paciente y período para visualizar.

8.2 Lo que aprendí

1. **Las matrices con punteros son lentas para acceder por posición:** Llegar a una celda (i,j) tarda O(i+j) desde la Cabeza, no O(1) como en arrays. Pero para obtener vecinos es súper rápido porque ya tengo los punteros.
2. **El histórico consume mucha memoria:** Guardar copias de matrices M×M en cada período se come harta RAM, sobre todo si M es grande o simulas muchos períodos. Pero no encontré otra forma de detectar patrones.
3. **Los genéricos ayudan bastante:** Hice Nodo, ListaEnlazada y Matriz solo una vez y los reutilicé para todo (int, Celda, Matriz, etc.).
4. **ASCII no sirve para matrices grandes:** Si M > 20 ya no se ve nada en consola. Tendría que haber usado algo gráfico.

8.3 Qué faltó o se puede mejorar

1. **Optimizar el acceso a celdas:** Tal vez usar una tabla hash personalizada para guardar referencias directas a cada nodo y que sea O(1).
2. **Ahorrar memoria en el histórico:** En vez de guardar matrices completas, solo guardar las diferencias entre períodos consecutivos.
3. **Hacer las reglas configurables:** Que el usuario pueda cambiar las condiciones de contagio/supervivencia desde el XML.
4. **Agregar Graphviz para generar imágenes:** Sería más fácil ver patrones complejos si se genera un archivo .dot y una imagen PNG de la rejilla.
5. **Procesar varios pacientes en paralelo:** Usar tareas asíncronas para analizar múltiples pacientes al mismo tiempo y que sea más rápido.

ANEXOS

Código Fuente

Todo el código está en GitHub:

- **Repo:** https://github.com/nova1809-xd/IPC2_Proyecto1_202300583

Archivos principales:

```
src/Proyecto1/
├── Nodo.cs
├── ListaEnlazada.cs
├── NodoMatriz.cs
├── Matriz.cs
├── Celda.cs
├── Paciente.cs
├── Automata.cs
└── Program.cs
```

Cómo Compilar y Ejecutar

Compilar

cd src/Proyecto1

dotnet build

Ejecutar:

cd bin/Debug/net10.0

./Proyecto1.exe

O directamente

dotnet run

Fecha: 5 de Marzo de 2026
Proyecto: IPC2 Primer Semestre 2025
Versión Final